Project Proposal: Secure Vault

A Virtual Encrypted Character Device Driver for Linux

Course: Operating Systems

Muhammad Rehan Siddiqui (24K-0707)
Abdullah Razzaq (24K-0691)
Ariz Ali (24K-1044)

March 19, 2026

1 Introduction / Background

In Unix-based systems, the "Everything is a File" philosophy allows users to interact with hardware and kernel services through file descriptors. A **Character Device Driver** is a kernel module that handles data as a stream of bytes. This project leverages that architecture to create a "Virtual Vault"—a location in the file system that looks like a file but performs real-time encryption.

2 Problem Statement

Standard user-space applications often store sensitive data in temporary buffers that are vulnerable to memory scraping or unauthorized access. There is a need for a kernel-level mechanism that abstracts the encryption process, ensuring that sensitive data is never stored in "plain text" within the designated device memory.

3 Objectives of the Project

- To develop a functional Linux Kernel Module (LKM).
- To implement secure data transfer between User Space and Kernel Space.
- To provide a transparent encryption/decryption layer using a symmetric cipher.
- To manage kernel memory safely without causing system instability.

4 Scope of the Project

The project focuses on the creation of a virtual device node (`/dev/vault`). It will support basic file operations: `open`, `read`, `write`, and `release`. The scope is limited to volatile storage (data persists only as long as the module is loaded).

5 Proposed System / Methodology

The system will be implemented as a C-based kernel module:

1. **Registration:** The driver will register a unique Major Number.
2. **Write Operation:** The driver intercepts data, applies a cipher, and stores it in a kernel-allocated buffer.
3. **Read Operation:** The driver decrypts the buffer and sends the original message back to the user.

6 Operating System Concepts Used

- **Kernel vs. User Space Isolation:** Managing restricted and privileged modes.
- **Virtual File System (VFS):** Using file abstraction for device I/O.
- **Memory Management:** Dynamic allocation using `kmalloc`.

7 Implementation Plan & Timeline

Week	**Task**
Week 1	Setup environment and "Hello World" LKM.
Week 2	Major/Minor number registration and node creation.
Week 3	Core logic: `read` and `write` implementation.
Week 4	Integration of encryption and final debugging.

8 Expected Results and Conclusion

The final result will be a `.ko` file. When loaded, users can securely store strings in the kernel. This project demonstrates how an OS manages data boundaries and hardware abstraction.